OSSP (Operating Systems And Systems Programming) - 25CS2104E

Roll No: 2520030106

Name: D. Mohit Venkat Sai

SEC-8

PRACTICAL-3:

Aim: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations

PROCEDURE :

1. **Running & Waiting:**

```
venkatsaikrishna5@VSK-HP:~$ mkdir practical3
venkatsaikrishna5@VSK-HP:~$ cd practical3
venkatsaikrishna5@VSK-HP:~/practical3$ nano process.c
venkatsaikrishna5@VSK-HP:~/practical3$ gcc process.c -o process
venkatsaikrishna5@VSK-HP:~/practical3$ ./process
Before fork():
PID  = 1349
PPID = 941


PARENT PROCESS
PID  = 1349
PPID = 941
Child PID = 1350
Parent is waiting for child...
CHILD PROCESS
PID  = 1350
PPID = 1349
Child is running...
Child is terminating...
Child terminated.
Parent is terminating...
venkatsaikrishna5@VSK-HP:~/practical3$ ps -o pid,ppid,state,cmd
    PID    PPID S CMD
    941     938 S -bash
   1442     941 R ps -o pid,ppid,state,cmd
```

2. **Monitoring:**

```
venkatsaikrishna5@VSK-HP:~/practical3$ top
top - 09:30:59 up 28 min,  1 user,  load average: 0.02, 0.05,
Tasks:  27 total,   1 running,  26 sleeping,   0 stopped,   0
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,
MiB Mem :   7796.6 total,   7142.5 free,    566.8 used,    238
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7229

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM
      1 root      20   0   24316  15380  11620 S   0.0   0.2
      2 root      20   0    3180   2208   2072 S   0.0   0.0
      6 root      20   0    3180   2044   1940 S   0.0   0.0
     49 root      19  -1   50424  16656  15384 S   0.0   0.2
     81 systemd+  20   0   22540  14580  11960 S   0.0   0.2
     88 root      20   0   35232  12316   9240 S   0.0   0.2
    150 root      20   0    2888   1948   1820 S   0.0   0.0
    151 root      20   0    4472   3132   2880 S   0.0   0.0
    152 message+  20   0    8792   5416   4736 S   0.0   0.1
    159 root      20   0   44092  29712  14572 S   0.0   0.4
    169 root      20   0   18432   9384   8144 S   0.0   0.1
    182 syslog    20   0  220548   5872   4604 S   0.0   0.1
    224 _chrony   20   0   24448  11848   7088 S   0.0   0.1
    231 _chrony   20   0   12092   2436   1792 S   0.0   0.0
    276 root      20   0    5492   2712   2524 S   0.0   0.0
    278 root      20   0  123456  32532  17924 S   0.0   0.4
    333 root      20   0    8648   5572   4728 S   0.0   0.1
    386 venkats+  20   0   22744  13304  10856 S   0.0   0.2
    390 venkats+  20   0   22912   4088   2088 S   0.0   0.1
    420 venkats+  20   0    6136   5424   3864 S   0.0   0.1
    936 root      20   0    3184   1120    980 S   0.0   0.0
    938 root      20   0    3200   1260   1108 S   0.0   0.0
    941 venkats+  20   0    6264   5540   3864 S   0.0   0.1
   1351 root      20   0    3184   1120    980 S   0.0   0.0
   1352 root      20   0    3200   1132    980 S   0.0   0.0
   1358 venkats+  20   0    6264   5536   3860 S   0.0   0.1
   1465 venkats+  20   0    8172   6008   3900 R   0.0   0.1
```

3. **Terminated:**

```
venkatsaikrishna5@VSK-HP:~/practical3$ ./process
Before fork():
PID  = 1577
PPID = 941


PARENT PROCESS
PID  = 1577
PPID = 941
Child PID = 1578
Parent is waiting for child...
CHILD PROCESS
PID  = 1578
PPID = 1577
Child is running...
ps -p 5834Child is terminating...
Child terminated.
Parent is terminating...
venkatsaikrishna5@VSK-HP:~/practical3$ ps -p 1578
    PID TTY          TIME CMD
```

Code:

```
  GNU nano 8.7.1                process.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Before fork():\n");
    printf("PID  = %d\n", getpid());
    printf("PPID = %d\n\n", getppid());

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
        return 1;
    }

    if (pid == 0)
    {
        printf("CHILD PROCESS\n");
        printf("PID  = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("Child is running...\n");

        sleep(20);

        printf("Child is terminating...\n");
        exit(0);
    }
    else
    {
```

```
    else
    {
        printf("\nPARENT PROCESS\n");
        printf("PID  = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("Child PID = %d\n", pid);

        printf("Parent is waiting for child...\n");

        wait(NULL);

        printf("Child terminated.\n");
        printf("Parent is terminating...\n");
    }

    return 0;
}
```

Observation :

- A parent process and a child process were created successfully using fork().
- The PID and PPID of both processes were displayed.
- The child process entered the waiting/sleeping (S) state during sleep().
- The parent process waited for the child using wait().
- Process information was successfully observed using ps, top, and /proc.
- After execution, both child and parent processes were terminated.

Result :

The experiment was successfully completed. The parent and child processes were created using fork(), and their PID, PPID, and process states were observed using Linux monitoring tools. The transitions between running, waiting, and terminated states were successfully verified.